Coding for AI

Program: **Repetition, Generators & Decorators**
Author: Tanishka Hira
Date: 24-10-2026

Decorator and generator

In [1]:
```python
import time
import functools


def log_execution(func):
    """Log the time taken to call a function."""

    @functools.wraps(func)
    def wrapper(*args, **kwargs):
        start_time = time.perf_counter()
        result = func(*args, **kwargs)
        elapsed_time = time.perf_counter() - start_time
        print(f"[LOG] Function '{func.__name__}' executed in {elapsed_time:.6f}s
        return result

    return wrapper


@log_execution
def stream_sensor_data(readings: list, threshold: float, /):
    """Yield numeric sensor readings greater than or equal to the threshold."""
    for reading in readings:
        if isinstance(reading, (int, float)) and reading >= threshold:
            yield reading
```

Telemetry pipeline

In [2]:
```python
raw_readings = [
    21.5, "ERR", 28.0, 14.2, None,
    31.0, "SENSOR FAIL", 18.5, 25.0
]
cutoff_threshold = 20.0

print("--- Starting Telemetry Pipeline ---")
filtered_readings = stream_sensor_data(raw_readings, cutoff_threshold)
fahrenheit_readings = (
    round(celsius * 1.8 + 32, 2)
    for celsius in filtered_readings
)

for fahrenheit in fahrenheit_readings:
    print(f"Processed Reading: {fahrenheit:.1f} F")
```

```
--- Starting Telemetry Pipeline ---
[LOG] Function 'stream_sensor_data' executed in 0.000001s
Processed Reading: 70.7 F
Processed Reading: 82.4 F
Processed Reading: 87.8 F
Processed Reading: 77.0 F
```

Verification

In [3]:
```python
assert stream_sensor_data.__name__ == "stream_sensor_data"
assert stream_sensor_data.__doc__
print("[+] Metadata introspection test passed successfully.")

try:
    stream_sensor_data(readings=raw_readings, threshold=cutoff_threshold)
except TypeError as error:
    print(f"TypeError: {error}")
```

```
[+] Metadata introspection test passed successfully.
TypeError: stream_sensor_data() got some positional-only arguments passed as keyw
ord arguments: 'readings, threshold'
```

keyword-only upper limit

In [4]:
```python
@log_execution
def stream_sensor_data(
    readings: list, threshold: float, /, *, upper_limit: float = 60.0
):
    """Yield numeric sensor readings within the specified temperature range."""
    for reading in readings:
        if isinstance(reading, (int, float)) and threshold <= reading <= upper_l
            yield reading


bonus_readings = [21.5, 999.9, 30.0]

print("Default upper limit (999.9 must be rejected):")
for reading in stream_sensor_data(bonus_readings, 20.0):
    print(reading)

print("\nCustom upper limit (999.9 must be accepted):")
for reading in stream_sensor_data(bonus_readings, 20.0, upper_limit=1000.0):
    print(reading)

print("\nThird positional argument (must raise TypeError):")
try:
    stream_sensor_data(bonus_readings, 20.0, 1000.0)
except TypeError as error:
    print(f"TypeError: {error}")
```

```
Default upper limit (999.9 must be rejected):
[LOG] Function 'stream_sensor_data' executed in 0.000002s
21.5
30.0

Custom upper limit (999.9 must be accepted):
[LOG] Function 'stream_sensor_data' executed in 0.000001s
21.5
999.9
30.0

Third positional argument (must raise TypeError):
TypeError: stream_sensor_data() takes 2 positional arguments but 3 were given
```

edge case experiments

In [5]:
```python
print("Boolean test:")
for reading in stream_sensor_data([True, False], 0.5):
    print(repr(reading))

print("\nSpecial floating-point values:")
for reading in stream_sensor_data(
    [float("nan"), float("inf"), float("-inf"), 25.0],
    20.0,
    upper_limit=float("inf")
):
    print(repr(reading))

print("\nOne-shot generator:")
one_shot = stream_sensor_data([22.0], 20.0)
for value in one_shot:
    print("First pass:", value)
for value in one_shot:
    print("Second pass:", value)
try:
    next(one_shot)
except StopIteration:
    print("next() on exhausted generator raised StopIteration")

print("\nAll-invalid stream:")
for reading in stream_sensor_data(["ERR", None, 12.0], 20.0):
    print("Unexpected reading:", reading)
print("Completed without errors.")
```

```
Boolean test:
[LOG] Function 'stream_sensor_data' executed in 0.000002s
True

Special floating-point values:
[LOG] Function 'stream_sensor_data' executed in 0.000002s
inf
25.0

One-shot generator:
[LOG] Function 'stream_sensor_data' executed in 0.000001s
First pass: 22.0
next() on exhausted generator raised StopIteration

All-invalid stream:
[LOG] Function 'stream_sensor_data' executed in 0.000001s
Completed without errors.
```

timing comparison

In [6]:
```python
large_readings = [25.0] * 2_000_000

print("Decorator timing (generator creation):")
large_stream = stream_sensor_data(large_readings, 20.0)

start_consumption = time.perf_counter()
count = 0
for _ in large_stream:
    count += 1
consumption_seconds = time.perf_counter() - start_consumption

print(f"Consumed {count:,} readings in {consumption_seconds:.6f}s")
print("Explanation: generator body execution is deferred until iteration.")
```

```
Decorator timing (generator creation):
[LOG] Function 'stream_sensor_data' executed in 0.000001s
Consumed 2,000,000 readings in 0.193313s
Explanation: generator body execution is deferred until iteration.
```

Reflection

In this practical, the decorator showed a very small execution time because it only measured the creation of the generator, not the actual processing. I tested the program with two million sensor readings. The time taken to process all readings was much higher because generators use lazy evaluation. This means values are processed one by one only when a `for` loop runs. To measure the actual filtering time, we should start the timer before the loop and stop it after all readings are processed. We can also modify the decorator to measure the complete iteration process instead of just creating the generator. Generators are useful because they save memory by avoiding unnecessary lists. They are especially helpful when working with large datasets or continuous sensor readings.